

## Network Models

OSI: Open Systems Interconnection (theoretical)

TCP/IP: practical implementation -- HTTP (application) -  
> TCP -> IP -> Ethernet (link)

| OSI Model       | TCP/IP           |
|-----------------|------------------|
| 7. Application  | Application      |
| 6. Presentation |                  |
| 5. Session      |                  |
| 4. Transport    | Transport        |
| 3. Network      | Internet/network |
| 2. Data link    | Link             |
| 1. Physical     |                  |

Encapsulation: data wrapped w/ headers at each layer

Raw IP is not used due to unreliability -> abstract complexities to TCP/UDP

**IP Packet Header:** up to 64KB (typically 1500 bytes)

- Fields: Version, TTL, Protocol (TCP/UDP), Source/Dest IP, Checksum

| TCP (Transmission Control Protocol)                                    | UDP (User Datagram Protocol)                                                |
|------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Connection-oriented (3-way handshake: SYN -> SYN-ACK -> ACK)           | Connectionless (no handshake, packets treated separately and independently) |
| Reliable (retransmits lost packets), no loss                           | Unreliable (no retransmission), "best-effort"                               |
| In-order delivery                                                      | No ordering guarantee                                                       |
| Higher latency, more overhead                                          | Lower latency, less overhead                                                |
| Stream-based (data as a continuous byte stream, no message boundaries) | Datagram-based (packet boundaries)                                          |
| Web (HTTP), email, file transfer                                       | DNS, VoIP, streaming                                                        |

**Addressing:** IP addresses identify a machine

Domain name system (DNS): converts names to addr;  
net.LookupIP(name) returns back slice

IPv4: 32-bit addr (four 8-bit octets), (192.168.1.10)

IPv6: 128-bit addr, (2001:0DB8:AC10:FE01::)

**Ports:** 16-bits to identify a program on a machine, 0-1023 is reserved for the OS, can use 1024-65535

127.0.0.1 (localhost), 169.254.x.x (link-local)

CIDR notation with /n as the prefix (n network bits)

- Nets described by variable-len prefix and length
- Arbitrary allocation between net and host address
- Pro: finer grained allocation; aggregation
- Con: more expensive lookup: longest prefix match

**Subnet mask:** n 1-bits (network bits followed by 32-n 0-bits (host bits))

**Network addr:** IP addr AND subnet mask

**Broadcast addr:** network addr + ( $2^{32-n}-1$ ), or net addr | (negated mask) (set all host bits to 1)

First usable host = network address + 1

Last usable host = broadcast address - 1

Number of usable addresses:  $2^{32-n} - 2$

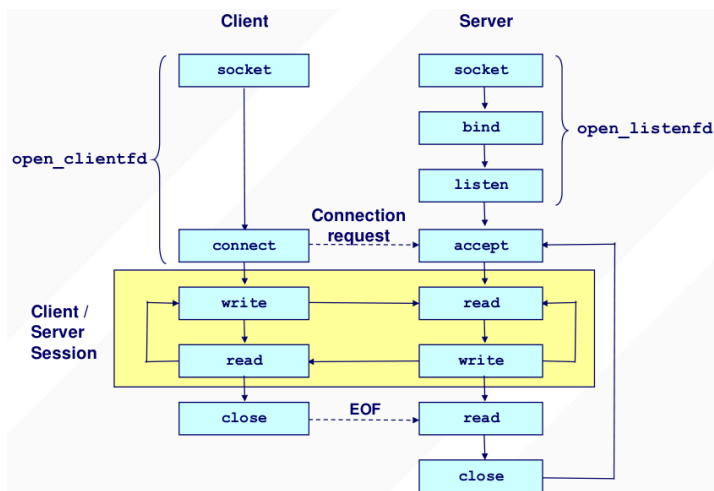
**Aggregation:** reduces router table size (e.g., 212.56.132.0/22 covers four /24 subnets, 4 subnets share /22 host bits)

**Example: Subnet 10.0.0.0/16 into four subnetworks**

- Need two bits for 4 subnets, so the new subnet mask is /18 (14 host bits)
- Each subnet has  $2^{32-18} = 16,384$  total addresses, with 16,382 usable
- Finding the start of each subnet: notice pattern in third octet change
  - Subnet 1: 00, 0 = 00000000
  - Subnet 2: 01, 64 = 01000000
  - Subnet 3: 10, 128 = 10000000
  - Subnet 4: 11, 192 = 11000000

**Socket:** an endpoint for communication between

machines (IP + Port) using system calls



**Endianness:** Use htons()/ntohs() to standardize to big endianness on the internet (network byte order)

send() blocks if SendQ is full, recv() blocks if RecvQ is empty -> design clear request/response roles

- Server is passive and waits for client to arrive
- Client has responsibility to send request once it arrives, server immediately starts
- Do not read/write at same time (will cause deadlock by both being blocked on read())

**Server:**

- Listen() (be passive) → Accept() → Spawn goroutine per connection
- Use defer conn.Close() for cleanup

**Client:**

- Dial() → Loop with Read()/Write()
- Handle partial reads with buffers/delimiters

```
// Dial TCP (client)
conn, err := net.Dial("tcp",
"localhost:8080")
conn.Write([]byte("hello"))
buf := make([]byte, 1024)
n, _ := conn.Read(buf)

// Listen TCP (server)
ln, err := net.Listen("tcp", ":8080")
for {
    conn, _ := ln.Accept()
    go handleConn(conn)
}

func handleConn(c net.Conn) {
    defer c.Close()
    buf := make([]byte, 1024)
```

```
n, _ := c.Read(buf)
c.Write([]byte("ack"))
}

// Main loop of a server
Remaining := ""
buf := make([]byte, 1024)
for {
    for "Does remaining contain a full
request?" {
        If yes, (1) parse it, then (2) remove
from remaining
    }
    size, err := c.Read(buf)
    data := buf[:size]
    remaining = remaining + string(data)
}
```

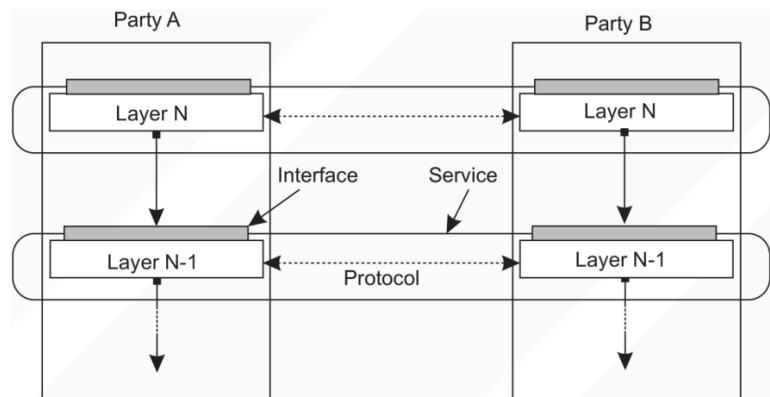
Service interface: what operations available to users

Protocol interface: how operations are encoded on the wire (formatting of the messages)

Specifies rules for comms. between similar layers:

Prose/BNF, Message Sequence Diagrams, State

Transition Diagrams, Packet Formats



Operation: action that can be performed in service inter.

Message: encoding of an operation or data according to a protocol's wire format (XML, binary, JSON, etc.)

Framing: structure of serializing data so messages can be separated and interpreted to/from stream; framing problem: determining message boundaries

- Option 1: length of each segment (security concerns about reading some random length)
- Option 2: delimiter (escape mechanism if the delimiter is within the message)

Parsing: encoding/decoding to/from application level data structure (i.e Golang struct to wire format)

---

### **Performance Metrics**

Bandwidth: # bits transmitted per unit time in a channel

Latency: Propagation+Transmit+Queue

- Propagation=Distance/Speedof Light(\*)
- Transmit = packet bits/Bandwidth
- Queue = Time waiting in traffic jams

Overhead: # secs for CPU to transmit

Error Rate: Probability p message will not arrive intact

Ping: test if other side of alive, measure round-trip latency: ping example.com

iperf3: time now long it takes to send N bytes (TCP/UDP throughput)

---

**Tail latency**: high-percentile delays in distributed systems (small amount of servers take a long time) overall latency delayed by the slowest response

Aggregation/front-end server: acts as “well-known” middle-man server for clients, fans out requests to backend servers; good for scalability

Large-scale online services: parallelize sub-operations, fan out request from root to leaf servers, then merge responses with request-distribution tree

With N servers, P(slow) per server ->  $P(\text{any slow}) = 1 - (1-P)^N$  (grows with more and more servers)

Percentiles: p50 = median, p99 = worst 1%; reduce 99% latency with techniques: hedging, reducing variability, and canary requests

Amplification: 5% slow servers → 50% latency impact

**Variability is dynamic**: mitigation is better than eliminating slower servers

Hardware trends: tolerating variability more important over time; higher bandwidth reduces per-message overheads (reduces costs of tied req.)

### **Reducing component variability**:

- Differentiating service classes
  - Differentiate non-interactive requests
  - Prioritize interactive over background tasks
- High level queuing: keep low level queues short
- Reduce head-of-line blocking
  - Break long-running requests into a sequence of smaller requests (request partitioning)
- Synchronize disruption
  - Do background activities altogether

### **Short-term (request) mitigation strategies:**

- **Hedged requests**: send duplicate request after a certain threshold, accept first response back
- Tied requests: send to multiple servers with cross-cancellation; once a server picks up the request, cancel the tied counterparts

### **Long-term adaptation mitigation strategies:**

- Load shedding: temporarily deprioritize slow nodes when a server exceeds a latency threshold; remove from being active, continue to send shadow traffic to monitor recovery
  - Micro-partitions: break data into multiple partitions to quickly redistribute across servers, process in parallel
  - Selective replication: replicate and distribute items that are detected/predicted to cause load imbalance
  - **Canary analysis**: route untested/new code paths to a small amount of leaf servers, only continue querying rest of servers if canaries returned within threshold
- 

Computer architecture: Mega  $2^{20}$ , Kilo  $2^{10}$

Computer networks: Mega  $10^6$ , Kilo  $10^3$

Mbps (nets: bits) vs. MBps (architecture: bytes)

Bandwidth: available over link (theoretical)

Throughput: available to application (actual)